

내 애플리케이션의 컨테이너화, AWS에서 시작하기

우수연 | IBM

SPEAKER

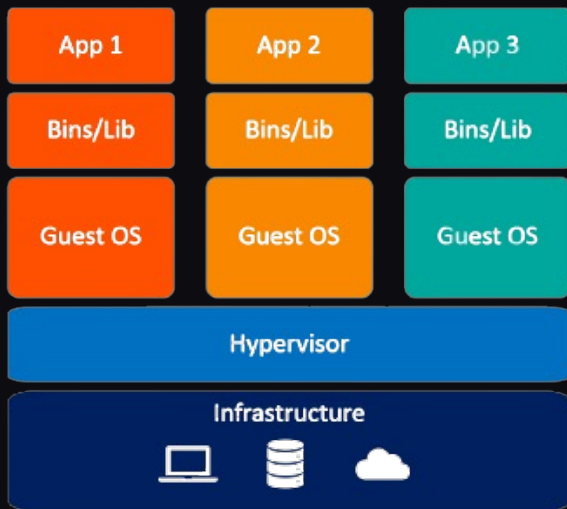
Suyeon Woo (우수연)

- Technology Engineer @IBM
- AWS Community Builder #containers
- AUSG 4th, Former Co-Organizer
(AWSKRUG University Student Group)
- GDG Cloud Korea Organizer
(Google Developer Group)

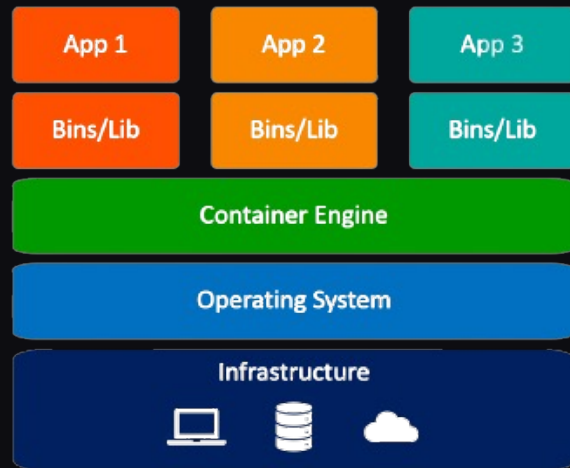
#Cloud #Modernization #Web3.0



Why Container?

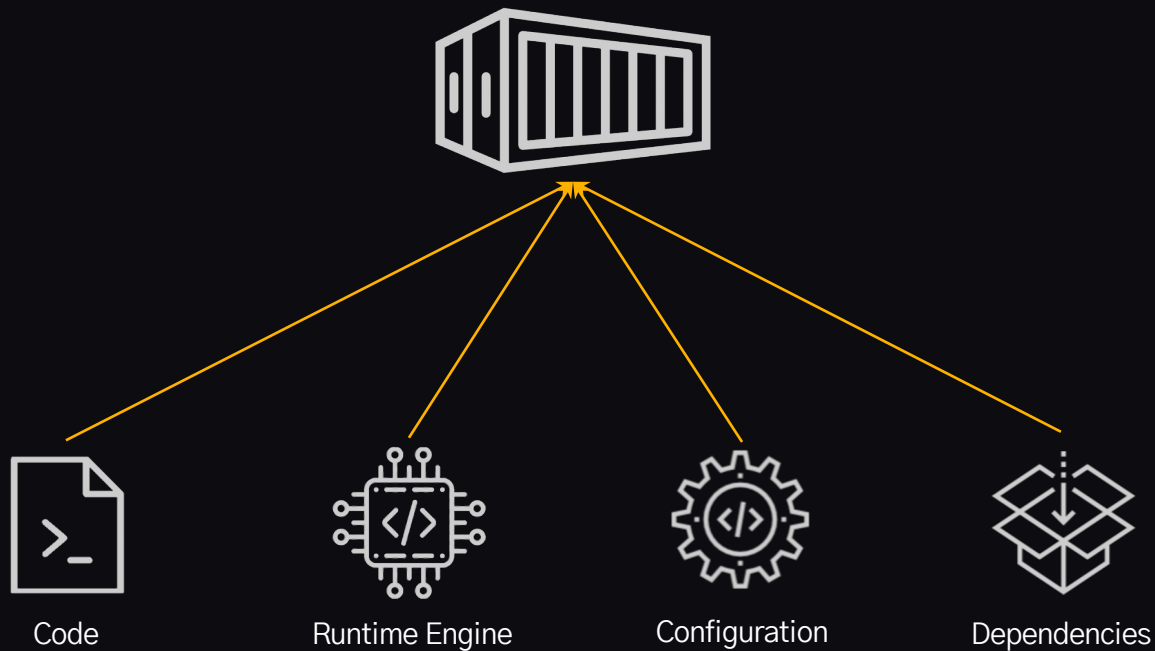


Virtual Machine

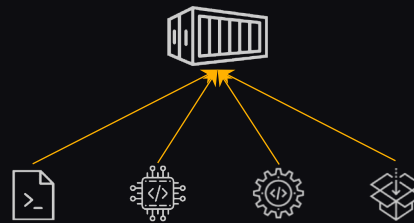
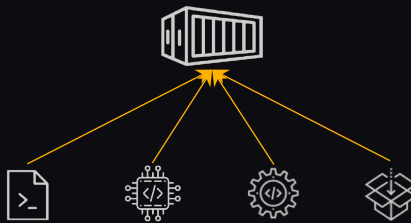
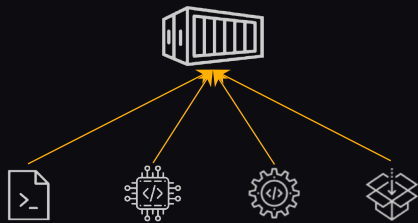
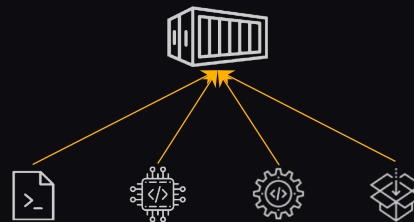
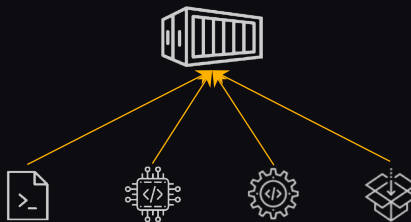
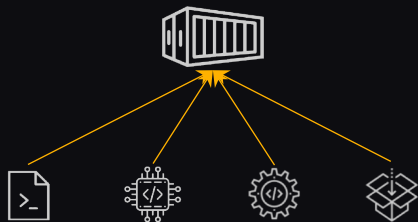


Container

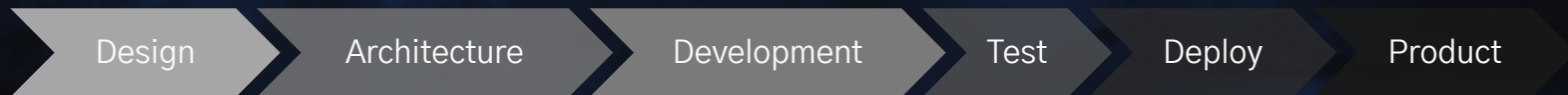
Why Container?



Why Container?



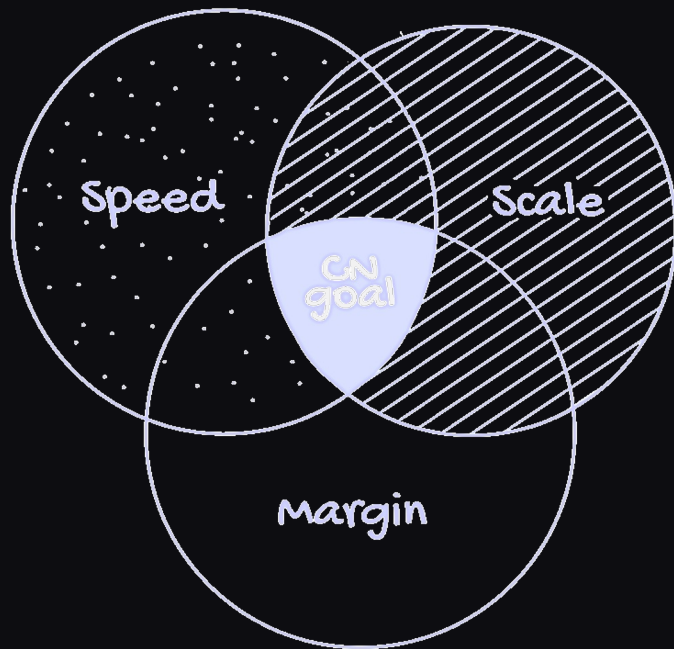
Why Container?



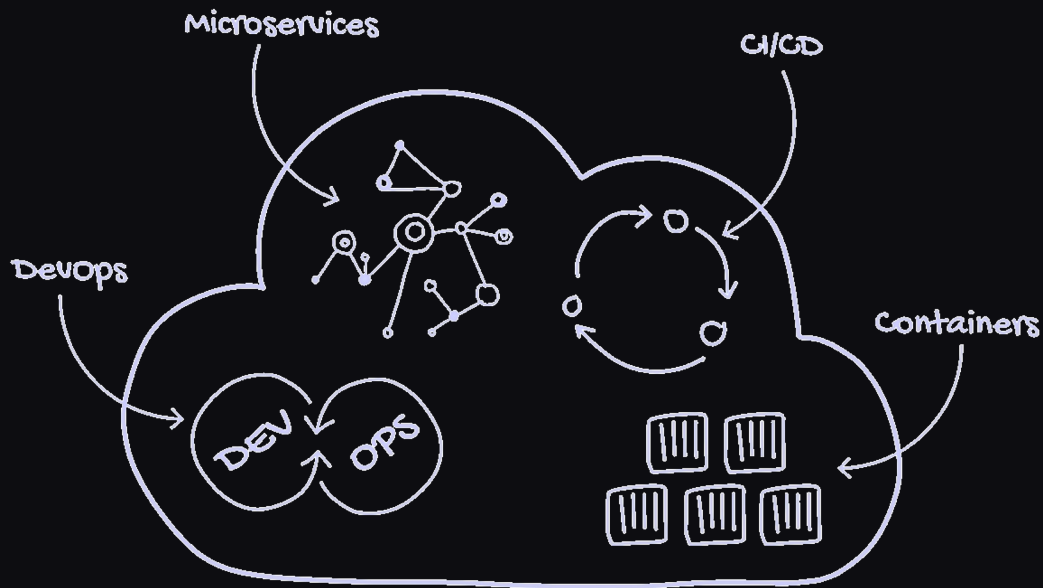
Use of **Cloud-Native** Technologies Will Be Pervasive, not Just Popular

By 2025, Gartner estimates that over **95%** of new digital workloads will be deployed on **cloud-native platforms**, up from 30% in 2021.

Cloud Native 목표

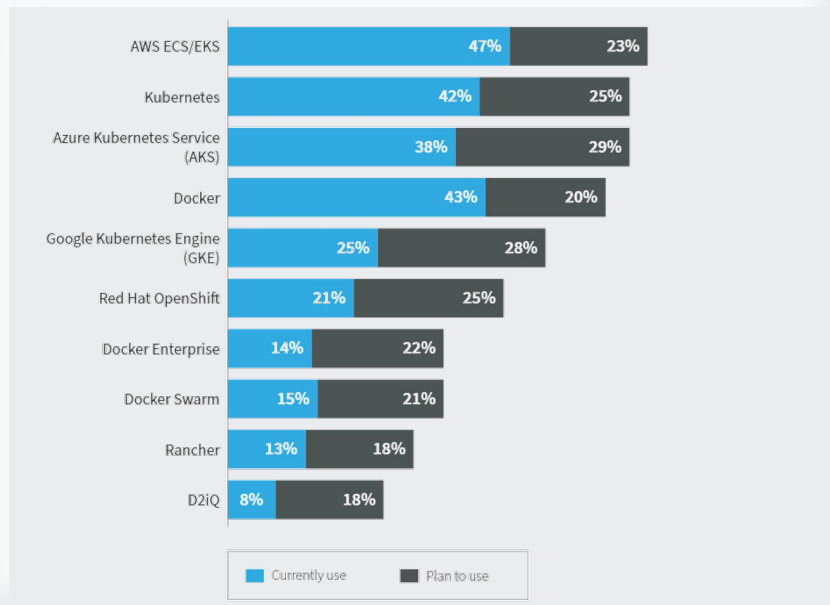


Cloud Native 주요 요소



Container use becomes increasingly mainstream

AWS ECS/EKS surpassed Docker for the top container spot this year.



Container Services in AWS

PaaS



Amazon Lightsail



AWS Elastic Beanstalk

Serverless



AWS Lambda



AWS App Runner



AWS Fargate

Orchestration



Amazon ECS



Amazon EKS

Enterprise



Amazon ECS Anywhere



Amazon EKS Anywhere



Red Hat OpenShift

내 어플리케이션의 컨테이너화

내 어플리케이션의 컨테이너화



AWS App2Container

Containerize and migrate existing applications

[Download AWS A2C](#)

Save on infrastructure and training costs by streamlining application development and operational skill sets.

Application 개발 및 운영 skill set을 간소화하여 인프라와 교육 비용 절감

Accelerate modernization with automatic analysis of applications and autogenerated container images.

Application과 컨테이너 이미지를 자동으로 분석하여 현대화 가속

Containerize applications running in your data center without code changes.

코드 변경 없이 데이터센터에서 실행중인 애플리케이션 컨테이너화

How it works

AWS App2Container (A2C) is a command line tool for migrating and modernizing Java and .NET web applications into container format. AWS A2C analyzes and builds an inventory of applications running in bare metal, virtual machines, Amazon Elastic Compute Cloud (EC2) instances, or in the cloud.

AWS App2Container(A2C)는 Java 및 .NET 웹 애플리케이션을 컨테이너 형식으로 마이그레이션하고 현대화하는 CLI tool 입니다. AWS A2C는 베어 메탈, 가상 머신, Amazon Elastic Compute Cloud(EC2) 인스턴스 또는 클라우드에서 실행되는 애플리케이션의 인벤토리를 분석하고 구축합니다.

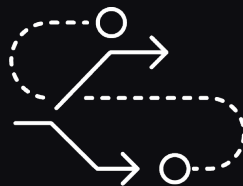
App2Container 작동 방식



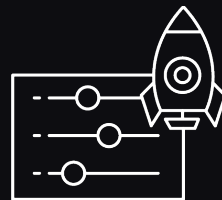
Discover and
Analyze



Extract and
Containerize



Create deployment
artifacts



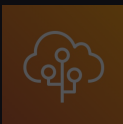
Deploy to AWS

Container Services for AWS App2Container

PaaS



Amazon Lightsail



AWS Elastic Beanstalk

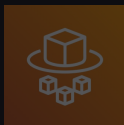
Serverless



AWS Lambda



AWS App Runner



AWS Fargate

Orchestration



Amazon ECS



Amazon EKS

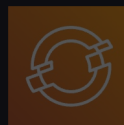
Enterprise



Amazon ECS Anywhere



Amazon EKS Anywhere



Red Hat OpenShift

Amazon ECS & EKS



Amazon ECS



Powerful Simplicity



Amazon EKS



Open flexibility

AWS App Runner



AWS App Runner provides a **highly abstracted** and **simple managed** experience for running web applications and API hosting services



Multi-concurrent



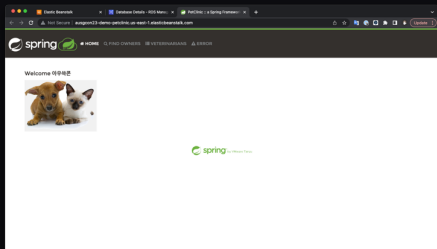
Stateless



No B.G. Processing

DEMO : AWS EB로 ec2에 배포되어 있는 자바 웹 애플리케이션

Java Application (Spring)



<https://github.com/spring-projects/spring-petclinic>



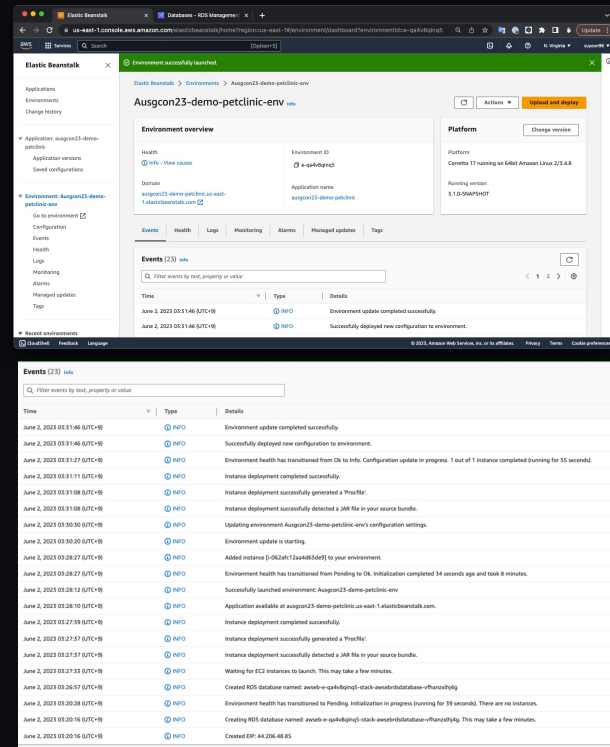
AS-IS : AWS Elastic Beanstalk Deployment



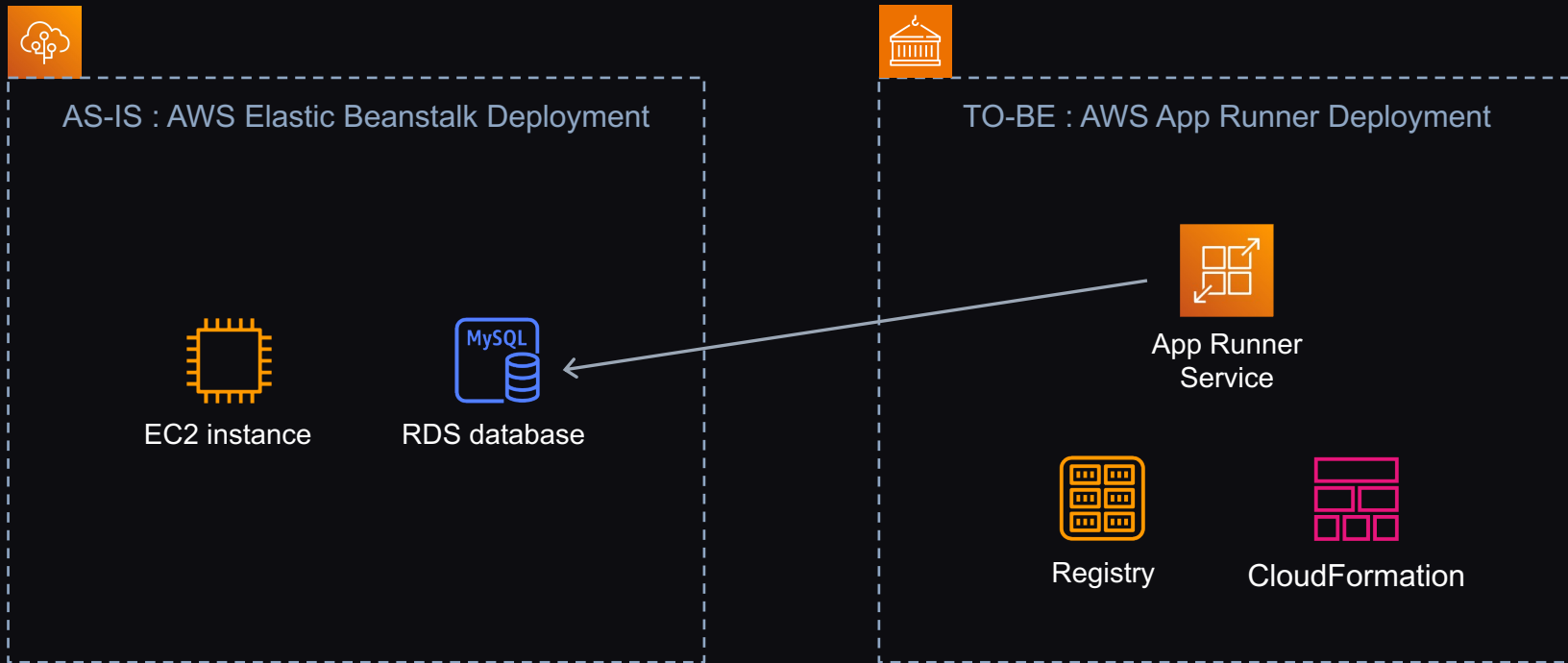
EC2 instance



RDS database



DEMO : AWS App2Container를 이용하여 App Runner로 컨테이너화

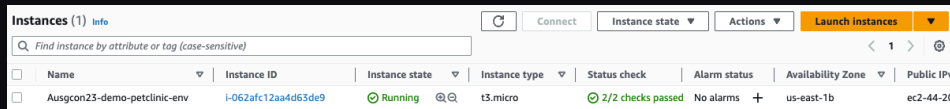


DEMO #0. Pre-requisites

1. AWS Account

2. Supported application type

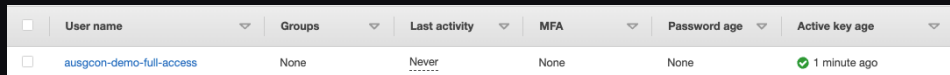
- ✓ Java application (Linux)
- ✓ ASP.NET application (Windows, Linux)



	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IP
☐	Ausccon23-demo-petclinic-env	i-062afc12aa4d63de9	Running	t3.micro	2/2 checks passed	No alarms	us-east-1b	ec2-44-20

3. Set up IAM resources for App2Container

- ✓ IAM resources for general use
- ✓ IAM resources for deployment



	User name	Groups	Last activity	MFA	Password age	Active key age
☐	ausccon-demo-full-access	None	Never	None	None	1 minute ago

DEMO #0. Pre-requisites

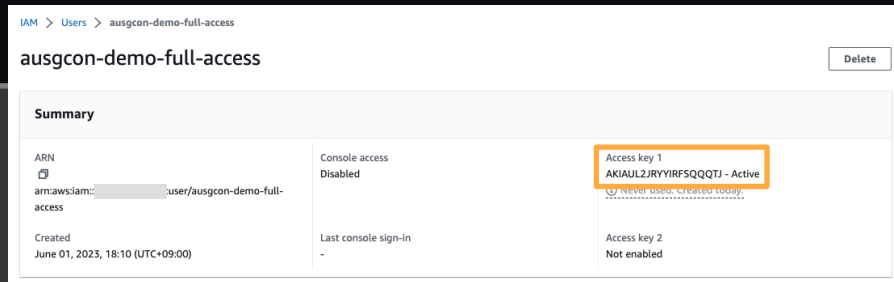
4. Install the AWS CLI

```
$ curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
$ unzip awscliv2.zip sudo
$ ./aws/install
```

5. Configure your AWS profile

```
$ aws configure
AWS Access Key ID [None]:
AWS Secret Access Key [None]:
Default region name [None]: us-east-1
Default output format [None]: json
```

```
$ aws sts get-caller-identity # Returns details about the IAM user or role
                             whose credentials are used to call the operation.
```



The screenshot shows the AWS IAM console page for a user named 'ausgcon-demo-full-access'. The page has a 'Delete' button in the top right corner. Below the user name, there is a 'Summary' section. The 'Summary' section contains a table with the following information:

Summary		
ARN	Console access	Access key 1
arn:aws:iam::[redacted]:user/ausgcon-demo-full-access	Disabled	AKIAUL2JRYIFFSQQQTJ - Active
Created	Last console sign-in	Access key 2
June 01, 2023, 18:10 (UTC+09:00)	-	Not enabled

The 'Access key 1' row is highlighted with an orange box. Below the ARN, there is a warning icon and the text 'Never used. Created today.'

DEMO #0. Pre-requisites

6. Install the Docker engine

```
# Installing Docker on Amazon Linux2
$ sudo yum update -y
$ sudo amazon-linux-extras install docker
$ sudo service docker start
$ sudo systemctl enable docker
$ sudo usermod -a -G docker ec2-user
```

suyeon96 — root@ip-172-31-95-100:~ — zsh (figterm) › ssh — 125x37

```
[[root@ip-172-31-95-100 ~]# docker info
Client:
 Context:    default
 Debug Mode: false
 Plugins:
  buildx: Docker Buildx (Docker Inc., 0.0.0+unknown)

Server:
 Containers: 0
  Running: 0
  Paused: 0
```

DEMO #1. Install App2Container

1. Install App2Container

```
suyeon96 — root@ip-172-31-95-100:~ — zsh (figterm) • ssh — 125x37

[root@ip-172-31-95-100 ~]# curl -o AWSApp2Container-installer-linux.tar.gz https://app2container-release-us-east-1.s3.us-east-1.amazonaws.com/latest/linux/AWSApp2Container-installer-linux.tar.gz
% Total    % Received % Xferd  Average Speed   Time    Time     Current
           Dload  Upload   Total   Spent    Left     Speed
100 18.9M  100 18.9M    0     0  25.5M      0 --:--:-- --:--:-- --:--:-- 25.5M
[root@ip-172-31-95-100 ~]#
[root@ip-172-31-95-100 ~]# ls
aws  AWSApp2Container-installer-linux.tar.gz  awscliv2.zip
[root@ip-172-31-95-100 ~]#
[root@ip-172-31-95-100 ~]# sudo tar xvf AWSApp2Container-installer-linux.tar.gz
uninstall.sh
install.sh
README
termsandconditions.txt
security/
AWSApp2Container.tar.gz
[root@ip-172-31-95-100 ~]#
[root@ip-172-31-95-100 ~]# sudo ./install.sh

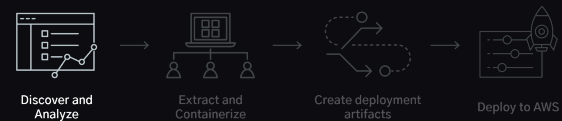
Determining user ...
Determining Installer Path ...
Checking the current installed A2C version
The AWS App2Container tool is licensed as "AWS Content" under the terms and conditions of the AWS Customer Agreement, located at https://aws.amazon.com/agreement and the Service Terms, located at https://aws.amazon.com/service-terms. By installing, using or accessing the AWS App2Container tool, you agree to such terms and conditions. The term "AWS Content" does not include software and assets distributed under separate license terms (such as code licensed under an open source license).
Do you accept the terms and conditions above? (y/n): y
Installing AWS App2Container ...
AWSApp2Container/
AWSApp2Container/resources/
```

DEMO #1. Install App2Container

2. Initialize App2Container

```
suyeon96 — root@ip-172-31-95-100:~ — zsh (figterm) • ssh — 125x37
[[root@ip-172-31-95-100 ~]# sudo app2container init
Workspace directory path for artifacts[default: /root/app2container]:
Use AWS EC2 Instance profile 'arn:aws:iam:::instance-profile/aws-elasticbeanstalk-ec2-role' configured with this
instance? (Y/N)[default: n]:
Optional AWS Profile (configured using 'aws configure --profile')[default: default]:
Optional S3 bucket for application artifacts:
Report usage metrics to AWS? (Y/N)[default: y]:
Automatically upload logs and App2Container generated artifacts on crashes and internal errors? (Y/N)[default: y]:
Require images to be signed using Docker Content Trust (DCT)? (Y/N)[default: n]:
Configuration saved
All application artifacts will be created under /root/app2container. Please ensure that the folder permissions are secure.
[root@ip-172-31-95-100 ~]#
```

DEMO #2. Discover and Analyze



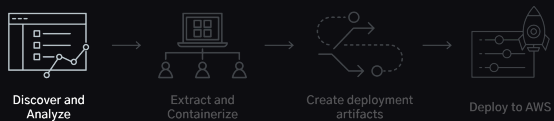
```
suyeon96 — root@ip-172-31-95-100:~ — zsh (figterm) › ssh — 125x37

[[root@ip-172-31-95-100 ~]# sudo app2container inventory
{
  "java-generic-3e2bdd2b" {
    "processId": 19437,
    "cmdline": "java -jar application.jar ",
    "applicationType": "java-generic",
    "webApp": ""
  }
}

[[root@ip-172-31-95-100 ~]#
[[root@ip-172-31-95-100 ~]# sudo app2container analyze --application-id java-generic-3e2bdd2b
✓ Created artifacts folder /root/app2container/java-generic-3e2bdd2b
✓ Generated analysis data in /root/app2container/java-generic-3e2bdd2b/analysis.json
👉 Analysis successful for application java-generic-3e2bdd2b

💡 Next Steps:
1. View the application analysis file at /root/app2container/java-generic-3e2bdd2b/analysis.json.
2. Edit the application analysis file as needed.
3. Start the containerization process using this command: app2container containerize --application-id java-generic-3e2bdd2b
[[root@ip-172-31-95-100 ~]#
[[root@ip-172-31-95-100 ~]#
```

DEMO #2. Discover and Analyze



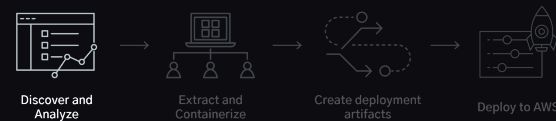
```
suyeon96 — root@ip-172-31-95-100:~ — zsh (figterm) › ssh — 125x37

[[root@ip-172-31-95-100 ~]# sudo app2container inventory
{
  "java-generic-3e2bdd2b": {
    "processId": 19437,
    "cmdline": "java -jar application.jar ",
    "applicationType": "java-generic",
    "webApp": ""
  }
}

[[root@ip-172-31-95-100 ~]#
[[root@ip-172-31-95-100 ~]# sudo app2container analyze --application-id java-generic-3e2bdd2b
✓ Created artifacts folder /root/app2container/java-generic-3e2bdd2b
✓ Generated analysis data in /root/app2container/java-generic-3e2bdd2b/analysis.json
👉 Analysis successful for application java-generic-3e2bdd2b

💡 Next Steps:
1. View the application analysis file at /root/app2container/java-generic-3e2bdd2b/analysis.json.
2. Edit the application analysis file as needed.
3. Start the containerization process using this command: app2container containerize --application-id java-generic-3e2bdd2b
[[root@ip-172-31-95-100 ~]#
[[root@ip-172-31-95-100 ~]# vi /root/app2container/java-generic-3e2bdd2b/analysis.json
```

DEMO #2. Discover and Analyze

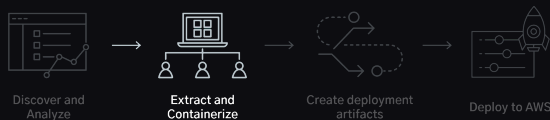


```
suyeon96 — root@ip-172-31-95-100:~ — zsh (figterm) • ssh — 125x37

{"a2CTemplateVersion": "1.0",
 "createdTime": "2023-06-01 19:14:171",
 "containerParameters": {
  "_comment1": "*** EDITABLE: The below section can be edited according to the application requirements. Please see the analysisInfo section below for details discovered regarding the application. ***",
  "imageRepository": "java-generic-3e2bdd2b",
  "imageTag": "latest",
  "containerBaseImage": "public.ecr.aws/amazonlinux/amazonlinux:2",
  "appExcludedFiles": [
   "/root/.aws"
  ],
  "appSpecificFiles": [],
  "applicationPort": 5000,
  "applicationMode": false,
  "logLocations": [],
  "enableDynamicLogging": false,
  "dependencies": []
 },
 "analysisInfo": {
  "_comment2": "*** NON-EDITABLE: Analysis Results ***",
  "processId": 19437,
  "appId": "java-generic-3e2bdd2b",
  "userId": "900",
  "groupId": "900",
  "cmdline": [
   "java",
   "-jar",
   "application.jar"
  ],
  "webApp": "",
  "osData": {
   "ANSI_COLOR": "0;33",
   "CPE_NAME": "cpe:2.3:o:amazon:amazon_linux:2",
   "HOME_URL": "https://amazonlinux.com/",
   "ID": "amzn",

```


DEMO #3. Extract and Containerize



```
suyeon96 — root@ip-172-31-95-100:~ — zsh (figterm) • ssh — 125x37
~ — root@ip-172-31-95-100:~ — zsh (figterm) • ssh
~ — root@ip-172-31-95-100:~ — zsh (figterm) • ssh

[[root@ip-172-31-95-100 ~]# sudo app2container containerize --application-id java-generic-3e2bdd2b
✓ Docker prerequisite check succeeded
✓ Tar prerequisite check succeeded
✓ Extracted container artifacts for application
✓ Entry file generated
✓ Dockerfile generated under /root/app2container/java-generic-3e2bdd2b/Artifacts
✓ Generated dockerfile.update under /root/app2container/java-generic-3e2bdd2b/Artifacts
✓ Generated deployment file at /root/app2container/java-generic-3e2bdd2b/deployment.json
✓ Deployment artifacts generated.
✓ Pre-validation succeeded.
👍 Containerization successful. Generated docker image java-generic-3e2bdd2b

💡 You're all set to test and deploy your container image.
```

Next Steps:

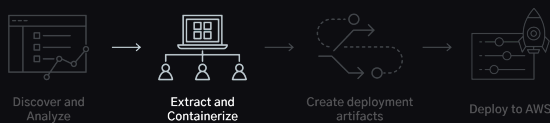
1. View the container image with "docker images" and test the application with "docker run --name java-generic-3e2bdd2b".
2. When you're ready to deploy to AWS, adjust the appropriate fields in /root/app2container/java-generic-3e2bdd2b/deployment.json to generate the desired deployment artifact. Note that by default "createEcsArtifacts" is set to true.
3. Generate deployment artifacts using "app2container generate app-deployment --application-id java-generic-3e2bdd2b".

```
[root@ip-172-31-95-100 ~]#
[root@ip-172-31-95-100 ~]# docker images
REPOSITORY                                TAG      IMAGE ID      CREATED        SIZE
java-generic-3e2bdd2b                     latest   28c9a99a13b2  2 minutes ago  2.7GB
public.ecr.aws/amazonlinux/amazonlinux    2        a18d1ca67c81  13 days ago   165MB

[root@ip-172-31-95-100 ~]#
[root@ip-172-31-95-100 ~]# cat /root/app2container/java-generic-3e2bdd2b/Artifacts/Dockerfile
FROM public.ecr.aws/amazonlinux/amazonlinux:2
MAINTAINER AWS
WORKDIR /
# Pre install packages
RUN yum -y install tar gzip shadow-utils.x86_64 wget
```

```
[root@ip-172-31-95-100 ~]# cat /root/app2container/java-generic-3e2bdd2b/Artifacts/Dockerfile
FROM public.ecr.aws/amazonlinux/amazonlinux:2
MAINTAINER AWS
WORKDIR /
# Pre install packages
RUN yum -y install tar gzip shadow-utils.x86_64 wget
# Copying Entryfile
COPY entryfile /entryfile
# Adding and unpacking Tar file
COPY ContainerFiles.tar /
RUN tar xvpf /ContainerFiles.tar --directory / --skip-old-files --same-owner --ignore-failed-read && rm -rf /ContainerFiles.t
as
# Follow the below example to update files
# COPY ["generic_config_file", "/root/app2container/java-generic-3e2bdd2b/Artifacts/generic_config_file"]
# User and user group
RUN groupadd -f -r -s 900 webapp
RUN id -u webapp & /dev/null 2>&1 || useradd -lrg 900 -u 900 webapp
# Environment Variables
ENV GRADE_HOME="/usr/local/grade"
ENV JAVA_HOME="/usr/lib/jvm/java-17-amazon-corretto.x86_64"
ENV LOGNAME="webapp"
ENV RDS_PASSWORD=""
ENV RDS_USERNAME="petclinic"
ENV SERVER_PORT="8080"
ENV SHELL="/bin/bash"
ENV HOME="/home/webapp"
ENV MZ_HOME="/usr/local/apache-maven"
ENV RDS_PORT="3306"
ENV PMD="/var/app/current"
ENV RDS_HOSTNAME="awsdb-e-q4v8qind5-stack-awsdbrcdsdatabase-vfnanzkhj4g.cqxcv483vari.us-east-1-rds.amazonaws.com"
ENV USER="webapp"
ENV _="/usr/bin/java"
ENV LANG="en_US.UTF-8"
ENV M2="/usr/local/apache-maven/bin"
ENV PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin"
ENV RDS_DB_NAME="todo"
ENV SHLV="1"
# Exposing ports
EXPOSE 8080
WORKDIR /var/app/current
# Stage Entrypoint
CMD /entryfile
# Health check on exposed port
HEALTHCHECK --interval=5s CMD wget --no-verbose --tries=1 --spider localhost:8080 || exit 1
```

DEMO #3. Extract and Containerize



```
[root@ip-172-31-95-100 ~]# vi /root/app2container/java-generic-3e2bdd2b/deployment.json
```

```
suyeon96 — root@ip-172-31-95-100:~ — zsh (figterm) · ssh — 125x55

{
  "a2cTemplateVersion": "1.0",
  "applicationId": "java-generic-3e2bdd2b",
  "imageName": "java-generic-3e2bdd2b",
  "exposedPorts": [
    {
      "localPort": 5000,
      "protocol": "tcp6"
    }
  ],
  "environment": [],
  "ecrParameters": {
    "ecrRepoTag": "latest"
  },
  "ecsParameters": {
    "createEksArtifacts": false,
    "ecsFamily": "java-generic-3e2bdd2b",
    "cpu": 2,
    "memory": 4096,
    "dockerSecurityOption": "",
    "publicApp": true,
    "stackName": "a2c-java-generic-3e2bdd2b-ECS",
    "resourceTags": [
      {
        "key": "example-key",
        "value": "example-value"
      }
    ],
    "reuseResources": {
      "vpcId": "",
      "reuseExistingA2cStack": {
        "cfnStackName": ""
      },
      "microserviceUrlPath": ""
    },
    "sshKeyPairName": "",
    "acmCertificateArn": ""
  },
  "gMSAParameters": {
    "domainSecretsArn": "",
    "domainDNSName": "",
    "domainNetBIOSName": "",
    "createGMSA": false,
    "gMSAName": ""
  },
  "deployTarget": "FARGATE",
  "dependentApps": []
},
  "fireLensParameters": {
    "enableFireLensLogging": false,
    "logDestinations": [
      {
        "service": "cloudwatch",
        "regexFilter": ".*\\.S",
        "streamName": "All-Logs"
      }
    ]
  },
  "eksParameters": {
    "createEksArtifacts": false,
    "cpu": 1.5,
    "memory": 3072,
    "stackName": "a2c-java-generic-3e2bdd2b-EKS",
    "reuseResources": {
      "vpcId": "",
      "reuseExistingA2cStack": {
        "cfnStackName": ""
      },
      "sshKeyPairName": "",
      "rootDomain": "",
      "acmCertificateArn": ""
    },
    "gMSAParameters": {
      "domainSecretsArn": "",
      "domainDNSName": "",
      "domainNetBIOSName": "",
      "createGMSA": false,
      "gMSAName": ""
    },
    "ingress": "alb",
    "dnsRecordName": "",
    "applicationPath": "/",
    "resourceTags": [
      {
        "key": "example-key",
        "value": "example-value"
      }
    ],
    "dependentApps": []
  },
  "appRunnerParameters": {
    "createAppRunnerArtifacts": true,
    "stackName": "a2c-java-generic-3e2bdd2b-AppRunner",
    "serviceName": "a2c-java-generic-3e2bdd2b",
    "autoDeploymentsEnabled": true,
    "resourceTags": [
      {
        "key": "example-key",
        "value": "example-value"
      }
    ]
  }
}
```

<https://docs.aws.amazon.com/app2container/latest/UserGuide/config-deployment.html>

DEMO #4. Create deployment artifacts



```
suyeon96 — root@ip-172-31-95-100:~ — zsh (figterm) • ssh — 125x37

[root@ip-172-31-95-100 ~]# vi /root/app2container/java-generic-3e2bdd2b/deployment.json
[root@ip-172-31-95-100 ~]# sudo app2container generate app-deployment --application-id java-generic-3e2bdd2b
✓ Docker prerequisite check succeeded
✓ AWS prerequisite check succeeded
✓ Created ECR repository 300265096721.dkr.ecr.us-east-1.amazonaws.com/java-generic-3e2bdd2b
✓ Pushed docker image 300265096721.dkr.ecr.us-east-1.amazonaws.com/java-generic-3e2bdd2b:latest to ECR repository
✓ Generated AWS App Runner CloudFormation template at /root/app2container/java-generic-3e2bdd2b/AppRunnerDeployment/apprunner.yml
👍 CloudFormation templates and additional deployment artifacts generated successfully for application java-generic-3e2bdd2b

💡 You're all set to use AWS CloudFormation to manage your application stack.

Next Steps:
1. Edit the CloudFormation template as necessary.
2. Create an application stack using the AWS CLI or the AWS Console. AWS CLI command:

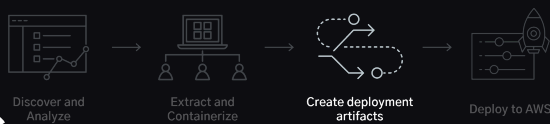
    aws cloudformation deploy --template-file /root/app2container/java-generic-3e2bdd2b/AppRunnerDeployment/apprunner.yml
    --capabilities CAPABILITY_IAM --stack-name a2c-java-generic-3e2bdd2b-AppRunner

3. Set up a pipeline for your application stack using app2container:

    app2container generate pipeline --application-id java-generic-3e2bdd2b

[root@ip-172-31-95-100 ~]#
```

DEMO #4. Create deployment artifacts



Amazon Elastic Container Registry

Private registry
Public registry
Repositories

Getting started ☒
Documentation ☒
Public gallery ☒

Amazon ECR > Repositories

Private Public

Private repositories (1)

☐ ☐ View push commands Delete Actions **Create repository**

☐	Repository name	URI	Created at	Tag immutability	Scan frequency	Encryption type	Pull through cache
☐	java-generic-3e2bdd2b	300265096721.dkr.ecr.us-east-1.amazonaws.com/java-generic-3e2bdd2b	June 02, 2023, 06:08:29 (UTC+09)	Disabled	Manual	AES-256	Inactive

Amazon Elastic Container Registry

Private registry
Public registry
Repositories
Summary
Images
Permissions
Lifecycle Policy
Repository tags

Elastic Beanstalk Database Details - RDS Man... Elastic Container Registry - Im... Step 5: Deploy your applica...

us-east-1.console.aws.amazon.com/ecr/repositories/private/300265096721/java-generic-3e2bdd2b?region=us-east-1

Amazon ECR > Repositories > java-generic-3e2bdd2b

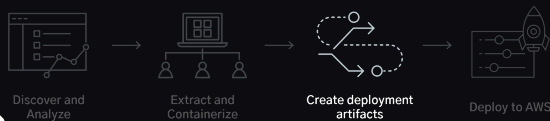
java-generic-3e2bdd2b

View push commands Edit

Images (1)

☐	Image tag	Artifact type	Pushed at	Size (MB)	Image URI	Digest	Scan status	Vulnerabilities
☐	latest	Image	June 02, 2023, 06:10:00 (UTC+09)	1233.36	Copy URI	sha256:db6b0aeb16ca9fd...	-	-

DEMO #4. Create deployment artifacts



CloudFormation console screenshot showing the **awseb-e-qa4v8qinq5-stack** details.

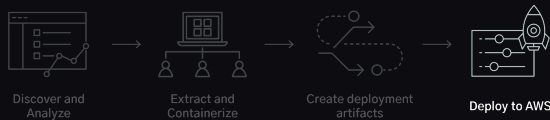
Stacks (1)

Stacks
awseb-e-qa4v8qinq5-stack 2023-06-02 03:19:44 UTC+0900 UPDATE_COMPLETE

Overview

Stack ID	Description
arn:aws:cloudformation:us-east-1:300265096721:stack/awseb-e-qa4v8qinq5-stack/e20b6f50-00a8-11ee-b6bf-122917926073	AWS Elastic Beanstalk environment (Name: 'Ausccon23-demo-petclinic-env' Id: 'e-qa4v8qinq5')
Status	Status reason
UPDATE_COMPLETE	-
Root stack	Parent stack
-	-
Created time	Deleted time
2023-06-02 03:19:44 UTC+0900	-
Updated time	
2023-06-02 03:52:39 UTC+0900	
Drift status	Last drift check time
NOT_CHECKED	-
Termination protection	IAM role
Deactivated	-

DEMO #5. Deploy to AWS



```
suyeon96 — root@ip-172-31-95-100:~ — zsh (figterm) • ssh — 125x37

💡 You're all set to use AWS CloudFormation to manage your application stack.

Next Steps:
1. Edit the CloudFormation template as necessary.
2. Create an application stack using the AWS CLI or the AWS Console. AWS CLI command:

    aws cloudformation deploy --template-file /root/app2container/java-generic-3e2bdd2b/AppRunnerDeployment/apprunner.yml
    --capabilities CAPABILITY_IAM --stack-name a2c-java-generic-3e2bdd2b-AppRunner

3. Set up a pipeline for your application stack using app2container:

    app2container generate pipeline --application-id java-generic-3e2bdd2b

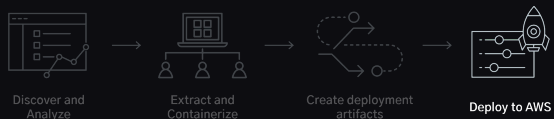
[root@ip-172-31-95-100 ~]# sudo app2container generate app-deployment --application-id java-generic-3e2bdd2b --deploy
✓ Docker prerequisite check succeeded
✓ AWS prerequisite check succeeded
✓ ECR repository 300265096721.dkr.ecr.us-east-1.amazonaws.com/java-generic-3e2bdd2b already exists
✓ Pushed docker image 300265096721.dkr.ecr.us-east-1.amazonaws.com/java-generic-3e2bdd2b:latest to ECR repository
✓ Generated AWS App Runner CloudFormation template at /root/app2container/java-generic-3e2bdd2b/AppRunnerDeployment/apprunner
.yml
✓ Updating CloudFormation stack
✓ Updating service
👍 Deployment successful for application java-generic-3e2bdd2b

Access your newly deployed app runner service at the following URL: https://xpepd9dqqq.us-east-1.awsapprunner.com
Stack deployed successfully!
Next Steps:

Set up a pipeline for your application stack using app2container:

    app2container generate pipeline --application-id java-generic-3e2bdd2b
```

DEMO #5. Deploy to AWS



us-east-1.console.aws.amazon.com/apprunner/home?region=us-east-1#/services

Services (1) Info

Filter services

Service name	Status	Default domain	Incoming traffic	Created	Last activity
a2c-java-generic-3e2bdd2b	Running	https://xpepd9dqqq.us-east-1... https://xpepd9dqqq.us-east-1.amazonaws.com	Public	6/1/2023, 9:16:29 PM UTC	6/1/2023, 9:16:29 PM UTC

App Runner > Services > a2c-java-generic-3e2bdd2b

a2c-java-generic-3e2bdd2b Info

Service overview

Status: Running

Incoming traffic: Public endpoint

Default domain: <https://xpepd9dqqq.us-east-1.amazonaws.com>

Service ARN: [arn:aws:apprunner:us-east-1:300265096721:service/a2c-java-generic-3e2bdd2b/c35c4f0f71ab410694f2696f6b2a9d0](#)

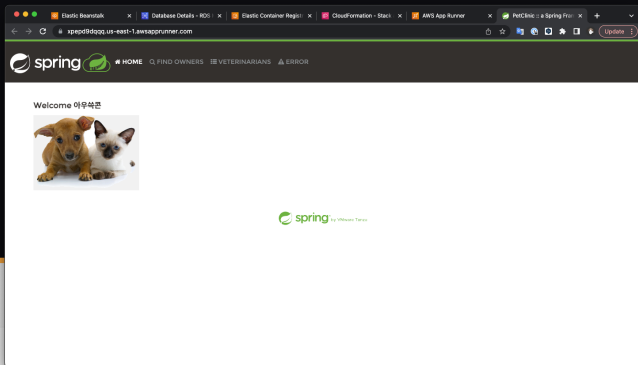
Source: [300265096721.dkr.ecr.us-east-1.amazonaws.com/java-generic-3e2bdd2b:latest](#)

Logs Activity Metrics Observability Configuration Custom domains

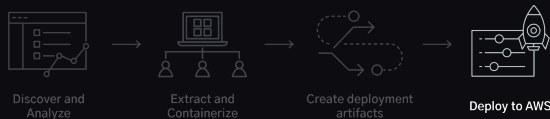
App Runner event logs Info

View logs of events in the lifecycle of your App Runner service.

```
1 06-02-2023 06:24:03 AM [AppRunner] Deployment with ID : abed881c6ba143:98a2f84c88c972a26 completed successfully.
2 06-02-2023 06:24:02 AM [AppRunner] Successfully routed incoming traffic to application.
3 06-02-2023 06:23:05 AM [AppRunner] Health check is successful. Routing traffic to application.
4 06-02-2023 06:21:34 AM [AppRunner] Performing health check on port "5000".
5 06-02-2023 06:21:24 AM [AppRunner] Provisioning instances and deploying image for publicly accessible service.
6 06-02-2023 06:21:14 AM [AppRunner] Successfully copied the image from ECR.
7 06-02-2023 06:17:47 AM [AppRunner] Successfully created pipeline for automatic deployments.
8 06-02-2023 06:17:11 AM [AppRunner] Deployment Artifact :- Repo Type: ECR; Image URL : 300265096721.dkr.ecr.us-east-1.amazonaws.com/java-generic-3e2bdd2b; Image Tag : latest
```



DEMO #5. Deploy to AWS



Browser tabs: Elastic Beanstalk, Database Details - RDS, Elastic Container Registry, CloudFormation - Stack, AWS App Runner, PetClinic :: a Spring Framework

Address bar: `xpepd9dqq.us-east-1.awsapprunner.com/owners?lastName=`

Navigation: HOME, FIND OWNERS, VETERINARIANS, ERROR

Owners

Name	Address	City	Telephone	Pets
Suyeon Woo	18F Centerfield	Seoul	6085551023	Leo
Thank You	AUSGCON	Seoul	6085551749	Basil
Eduardo Rodriguez	2693 Commerce St.	McFarland	6085558763	Jewel, Rosy
Harold Davis	563 Friendly St.	Windsor	6085553198	Iggy
Peter McTavish	2387 S. Fair Way	Madison	6085552765	George

Pages: [1 2]



Output petclinic.owners

	id	first_name	last_name	address	city	telephone
1	1	Suyeon	Woo	18F Centerfield	Seoul	6085551023
2	2	Thank	You	AUSGCON	Seoul	6085551749
3	3	Eduardo	Rodriguez	2693 Commerce St.	McFarland	6085558763
4	4	Harold	Davis	563 Friendly St.	Windsor	6085553198
5	5	Peter	McTavish	2387 S. Fair Way	Madison	6085552765
6	6	Jean	Coleman	105 N. Lake St.	Monona	6085552654
7	7	Jeff	Black	1450 Oak Blvd.	Monona	6085553387
8	8	Maria	Escobito	345 Maple St.	Madison	6085557683
9	9	David	Schroeder	2749 Blackhawk Trail	Madison	6085559435
10	10	Carlos	Estaban	2335 Independence La.	Waunakee	6085555487

Thank you for listening